In [1]:
```
from pyspark.sql import SparkSession
```

In [2]:
```
spark = SparkSession.builder \
    .appName("pyspark") \
    .getOrCreate()
```

In [3]:
```
from pyspark.sql.functions import col,sum,rank,dense_rank,udf
from pyspark.sql.window import Window
from pyspark.sql.types import StringType
```

Load all 3 datasets into DataFrames

In [4]:
```
customers_df = spark.read.csv(
    "customers.csv",
    header=True,
    inferSchema=True
)

orders_df = spark.read.csv(
    "orders.csv",
    header=True,
    inferSchema=True
)

order_items_df = spark.read.csv(
    "order_items.csv",
    header=True,
    inferSchema=True
)
```

Display schema

In [5]:
```
customers_df.printSchema()
orders_df.printSchema()
order_items_df.printSchema()
```

```
root
 |-- customer_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- age: integer (nullable = true)

root
 |-- order_id: integer (nullable = true)
 |-- customer_id: integer (nullable = true)
 |-- order_date: date (nullable = true)

root
 |-- order_id: integer (nullable = true)
 |-- product: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: integer (nullable = true)
 |-- quantity: integer (nullable = true)
```

Sample data

In [6]:
```
customers_df.show(5)
orders_df.show(5)
order_items_df.show(5)
```

```
+-----------+-------+--------+---+
|customer_id|   name|    city|age|
+-----------+-------+--------+---+
|        101|  Alice|New York| 34|
|        102|    Bob| Chicago| 28|
|        103|Charlie|New York| 41|
|        104|  David| Seattle| 36|
|        105|    Eva| Chicago| 30|
+-----------+-------+--------+---+
only showing top 5 rows
+--------+-----------+----------+
|order_id|customer_id|order_date|
+--------+-----------+----------+
|    1001|        101|2024-01-10|
|    1002|        102|2024-01-12|
|    1003|        103|2024-01-15|
|    1004|        101|2024-02-01|
|    1005|        104|2024-02-03|
+--------+-----------+----------+
only showing top 5 rows
+--------+-------+-----------+-----+--------+
|order_id|product|   category|price|quantity|
+--------+-------+-----------+-----+--------+
|    1001| Laptop|Electronics|  800|       1|
|    1001|  Mouse|Electronics|   20|       2|
|    1002|  Phone|Electronics|  500|       1|
|    1003|   Desk|  Furniture|  200|       1|
|    1003|  Chair|  Furniture|  120|       2|
+--------+-------+-----------+-----+--------+
only showing top 5 rows
```

Count number of records in each dataset

In [7]:
```
print("Customers Count :",customers_df.count())
print("Orders Count :",orders_df.count())
print("Order Items Count :",order_items_df.count())
```

```
Customers Count : 8
Orders Count : 10
Order Items Count : 12
```

Filter orders where total_amount > 1000 and add to a new column

In [8]:
```
order_items_df=order_items_df.withColumn(
    "total_amount",
    col("price")*col("quantity")
)

order_total_df=order_items_df.groupBy("order_id")\
               .agg(sum("total_amount").alias("total_amount"))

order_total_df.filter(col("total_amount")>1000).show()
```

```
+--------+------------+
|order_id|total_amount|
+--------+------------+
+--------+------------+
```

Perform joins to create a final enriched dataset

👉 Steps:

1. Join orders with customers
2. Join the result with order_items

👉 Final columns:

- order_id
- customer_name
- city
- product
- category
- total_amount

In [9]:
```
order_customer_df=orders_df.join(
    customers_df,
    on="customer_id",
    how="inner"
)
final_df=order_customer_df.join(
    order_items_df,
    on="order_id",
    how="inner"
).select(
    "order_id",
    col("name").alias("customer_name"),
    "city",
    "product",
    "category",
    "total_amount"
)

final_df.columns
```

Out[9]:
```
['order_id', 'customer_name', 'city', 'product', 'category', 'total_amount']
```

Rank customers by total spending in each city using window functions

In [10]:
```
customer_spending_df=final_df.groupBy("customer_name","city")\
                    .agg(sum("total_amount").alias("total_spending"))

city_window=Window.partitionBy("city").orderBy(col("total_spending").desc())

ranked_customers_df=customer_spending_df.withColumn(
    "rank",
    rank().over(city_window)
)

ranked_customers_df.show()
```

```
+-------------+--------+--------------+----+
|customer_name|    city|total_spending|rank|
+-------------+--------+--------------+----+
|        Helen|  Boston|           300|   1|
|        Grace|  Boston|            50|   2|
|          Bob| Chicago|           750|   1|
|          Eva| Chicago|           300|   2|
|        Alice|New York|          1440|   1|
|      Charlie|New York|           440|   2|
|        Frank| Seattle|          1000|   1|
|        David| Seattle|           700|   2|
+-------------+--------+--------------+----+
```

Find top 2 most expensive orders per customer

In [11]:
```
customer_order_total_df=final_df.groupBy("customer_name","order_id")\
                    .agg(sum("total_amount").alias("order_total"))

customer_window=Window.partitionBy("customer_name").orderBy(col("order_total").desc())

top_orders_df=customer_order_total_df.withColumn(
    "rank",
    dense_rank().over(customer_window)
)

top_orders_df.filter(col("rank")<=2).show()
```

```
+-------------+--------+-----------+----+
|customer_name|order_id|order_total|rank|
+-------------+--------+-----------+----+
|        Alice|    1001|        840|   1|
|        Alice|    1004|        600|   2|
|          Bob|    1002|        500|   1|
|          Bob|    1010|        250|   2|
|      Charlie|    1003|        440|   1|
|        David|    1005|        700|   1|
|          Eva|    1006|        300|   1|
|        Frank|    1007|       1000|   1|
|        Grace|    1008|         50|   1|
|        Helen|    1009|        300|   1|
+-------------+--------+-----------+----+
```

In [12]:
```
final_df.createOrReplaceTempView("sales")
```

Spark SQL : Find top 2 most expensive orders per customer

In [13]:
```
spark.sql("""
          select * from
          (
            select *,
            dense_rank() over (partition by customer_name order by order_total desc) as rank
            from (
                  select
                  customer_name, order_id,
                  sum(total_amount) as order_total
                  from sales
                  group by customer_name,order_id
            )
          )
```

```
          where rank<=2
""").show()
```

```
+-------------+--------+-----------+----+
|customer_name|order_id|order_total|rank|
+-------------+--------+-----------+----+
|        Alice|    1001|        840|   1|
|        Alice|    1004|        600|   2|
|          Bob|    1002|        500|   1|
|          Bob|    1010|        250|   2|
|      Charlie|    1003|        440|   1|
|        David|    1005|        700|   1|
|          Eva|    1006|        300|   1|
|        Frank|    1007|       1000|   1|
|        Grace|    1008|         50|   1|
|        Helen|    1009|        300|   1|
+-------------+--------+-----------+----+
```

Spark SQL : Top 3 customers by spending

In [14]:
```
spark.sql("""
          select customer_name,
          sum(total_amount) as total_spending
          from sales
          group by customer_name
          order by total_spending desc
          limit 3
""").show()
```

```
+-------------+--------------+
|customer_name|total_spending|
+-------------+--------------+
|        Alice|          1440|
|        Frank|          1000|
|          Bob|           750|
+-------------+--------------+
```

Spark SQL : Category-wise revenue

In [15]:
```
spark.sql("""
          select category,
          sum(total_amount) as revenue
          from sales
          group by category
          order by revenue desc
""").show()
```

```
+-----------+-------+
|   category|revenue|
+-----------+-------+
|Electronics|   3590|
|  Furniture|   1390|
+-----------+-------+
```

Classify customers into segments based on total spending using a UDF

Add a new column Segment and populate the data

In [16]:
```
customer_spending_df=final_df.groupBy("customer_name")\
                     .agg(sum("total_amount").alias("total_spending"))

def customer_segment(spending):
    if spending>=1500:
        return "Platinium"
    elif spending>=1000:
        return "Gold"
    elif spending>=500:
        return "Silver"
    else:
        return "Bronze"

segment_udf = udf(customer_segment, StringType())

customer_spending_df.withColumn("segment",segment_udf(col("total_spending"))).show()
```

```
+-------------+--------------+-------+
|customer_name|total_spending|segment|
+-------------+--------------+-------+
|        Grace|            50| Bronze|
|          Eva|           300| Bronze|
|        Helen|           300| Bronze|
|      Charlie|           440| Bronze|
|          Bob|           750| Silver|
|        Alice|          1440|   Gold|
|        David|           700| Silver|
|        Frank|          1000|   Gold|
+-------------+--------------+-------+
```